DB10

INSERT

> **Table에 새로운 Tuple을 추가하는 명령어**

- Table instance를 변경한다.
- `Table instance` : 특정 시점에 테이블에 실제로 들어있는 데이터의 집합

하나의 Tuple만을 추가하는 경우

INSERT INTO <table name> (<list of attributes>)

VALUES <Attribute values list>;

<list of attributes>를 생략하는 경우

- <Attribute values list>: **CREATE TABLE에서 입력한 Table의 attribute 순서, 개수, 대응되는 Type이 동일해야 한다.**
- 모든 Attribute에 대응되는 값들이 입력되어야 한다.

<list of attributes>를 생략하지 않는 경우

- **NULL 값을 가질 수 있거나 DEFUALT 값이 설정되어져 있는 Attribute**는 제외할 수 있다.
    - 이 경우, 해당 Attribute는 NULL 또는 DEFUALT 값을 갖게 된다.

여러 Tuple을 한번에 추가하는 경우

보통 **SELECT 문을 통해 Retreive한 결과를 Insert하는 경우**이다.

INSERT INTO <table name>(<list of attributes>) <select statement>;

- Select된 Attribute의 개수, 순서, Type이 (<list of attributes>) 와 동일해야 한다.

<list of attributes>를 생략할 수도 있는데, 이 경우 <select> 문의 결과가 Table에 정의된 모든 Column 순서, 개수, Type과 일치해야 한다.

직접 여러 Tuple을 추가할 수도 있다.

INSERT INTO <table name> (<list of attributes>)

VALUES

<Attribute values list>

<Attribute values list>

<Attribute values list>;

SELECT Statement를 통한 INSERT로 Table 생성 시 문제점

INSERT INTO <table name>(<list of attributes>) <select statement>;를 통해 INSERT할 경우, 그렇게 생성된 Table은 **up-to-date**하지 않다는 문제점이 있다.

- <Select statement>에서 사용된 Table들의 값이 변화해도 <table_name>에 해당되는 table의 값이 자동으로 update되지 않는다.

여러 Base table을 Join해서 결과 Tuple 집합을 한 번에 INSERT하여 Derived table을 생성하고 직접 저장했을 때, Base table의 변화가 Derived table에 자동으로 Update되지 않는다.

자동으로 Update하도록 하기 위해선 **VIEW**을 사용해야 한다.

DELETE

> **Tuple을 Table에서 제거하는 명령어**

삭제할 Tuple을 찾는 조건을 명시하게 위해 **WHERE clause**를 사용한다.

한번에 하나의 Table에서만 Tuple을 삭제할 수 있다.

DELETE FROM <table name>
WHERE <condition>;

WHERE이 없다면 Table 내의 모든 Tuple을 제거한다.

- **Table을 자체를 삭제하는 건 아니다.**

UPDATE

> **Tuple의 Attribute value를 변경하는 명령어**

- 여러 Attribute를 한 번에 변경할 수 있다.

업데이트 할 값을 명시하기 위해 **SET clasuse**가 사용된다.

변경할 Tuple을 찾는 조건을 명시하게 위해 **WHERE clause**를 사용한다.

한번에 하나의 Table에서만 Tuple을 변경할 수 있다.

UPDATE <table name>

SET <list of attribute = value pair>
WHERE <condition>

WHERE이 없다면 Table 내의 모든 Tuple의 해당 Attribute를 변경한다.

변경할 값을 **Arithmetic operator**를 이용하여 정의할 수 있다.

- 이 경우에는 **수정 전 Old value를 참조하여 New value가 계산**된다.

```
UPDATE    EMPLOYEE
SET       Salary = Salary * 1.1
WHERE     Dno = 5;
```

View

다른 Table로부터 유도되는 하나의 Table

물리적으로 저장되는 Table이 아니고 **Virtual table**이다.

- VIEW는 **SELECT 정의만 저장**하고 **실제 Tuple을 물리적으로 저장하지 않는다.**
- VIEW에 Query가 주어질 때, 미리 정의되어 있는 SELECT문을 통해 여러 Table로부터 가져오는 방법이다.
- 물리적으로 저장되지 않기 때문에 **UPDATE에는 제한이 있다.**

VIEW에 Query를 하는 것에는 **어떠한 제약도 없다**.

- Query가 주어지면 그때 Base table로부터 Tuple을 가져와서 적용하기 때문이다.
    - 이때에도 실제로 테이블을 구성하는 것은 아니고 각각 Tuple들만 끌어와 사용한다.

VIEW를 다루는 명령어는 DDL이다.

CREATE VIEW

CREATE VIEW <view name>

AS <select statement>

기본적으로 **CREATE VIEW**를 통해 생성된 **VIEW의 Logical한 Attribute name은 SELECT statement 결과의 Attribute name과 동일**하다.

```
CREATE VIEW    WORKS_ON1
AS SELECT      Fname, Lname, Pname, Hours
    FROM       EMPLOYEE, PROJECT, WORKS_ON
    WHERE      Ssn=Essn AND Pno=Pnumber;
```

CREATE VIEW <view name>(attribute name) **AS** <select statement> 로 **attribute name을 지정**할 수도 있다.

```
CREATE VIEW    DEPT_INFO(Dept_name, No_of_emps, Total_sal)
AS SELECT      Dname, COUNT (*), SUM (Salary)
    FROM       DEPARTMENT, EMPLOYEE
    WHERE      Dnumber=Dno
    GROUP BY   Dname;
```

Querying VIEW

1. VIEW가 Table인 것처럼 Query를 작성

- FROM 절에 view_name을 사용한다.

2. VIEW에 대해 Query가 들어오면 **DBMS는 먼저 SELECT statement를 실행하여 결과 Tuple들을 계산**
3. 결과 Tuple들을 Table의 Tuple처럼 생각하여 (1)의 Query를 처리

DROPING VIEW

더 이상 필요하지 않은 VIEW를 제거하는 명령어

DROP VIEW <view name>;

UPDATING VIEW

VIEW는 Physical table로 존재하지 않기 때문에 **Underlying (Base table)에 대한 UPDATE**로 이해할 수 있다.

특정 VIEW에 대한 Update는 가능한 경우가 있다.

- 정확한 매핑이 존재하는 경우

그러나 **Ambiguous한 경우에는 UPDATE가 불가능**하다.

Example 1

아래와 같은 VIEW를 생각해보자.

```
CREATE VIEW    WORKS_ON1
AS SELECT      Fname, Lname, Pname, Hours
    FROM       EMPLOYEE, PROJECT, WORKS_ON
    WHERE      Ssn=Essn AND Pno=Pnumber;
```

이후, 사용자가 아래와 같은 Query를 작성했다고 해보자.

- 사용자는 WORKS_ON1이 실제 Table인지 VIEW인지 모른 상태로 Query를 작성한다.
- **원래 의도**: Update the PNAME attribute of `John Smith' from 'ProductX' to `ProductY'

```
UPDATE WORKS_ON1
SET        Pname = 'ProductY'
WHERE      Lname='Smith' AND Fname='John'
           AND Pname='ProductX';
```

이때, **두 가지 가능성**이 생긴다.

1. **PROJECT Table**의 Pname을 업데이트
   - John smith가 일하는 Project의 이름을 변경하는 것

```
UPDATE PROJECT    SET    Pname = 'ProductY'
WHERE      Pname = 'ProductX';
```

2. **WORKS_ON Table**의 Pname을 업데이트
   - 해당 Employee가 일하는 Project 자체를 변경하는 것

```
UPDATE WORKS_ON
SET        Pno =    ( SELECT   Pnumber
                     FROM     PROJECT
                     WHERE    Pname='ProductY' )
WHERE      Essn IN  ( SELECT   Ssn
                     FROM     EMPLOYEE
                     WHERE    Lname='Smith' AND Fname='John' )
           AND
           Pno =    ( SELECT   Pnumber
                     FROM     PROJECT
                     WHERE    Pname='ProductX' );
```

DBMS의 입장에선 이 경우 1번과 2번 중 어떤 UPDATE 방식을 선택할 지 모른다.

- Query를 작성한 사람의 의도를 모르기 때문이다.
- 이 경우 **UPDATE를 처리**하지 않는다.

Example 2

아래와 같은 VIEW를 생각해보자.

```
CREATE VIEW    DEPT_INFO(Dept_name, No_of_emps, Total_sal)
AS SELECT      Dname, COUNT (*), SUM (Salary)
    FROM       DEPARTMENT, EMPLOYEE
    WHERE      Dnumber=Dno
    GROUP BY   Dname;
```

이때, 아래와 같은 QUERY가 들어온다.

```
UPDATE DEPT_INFO
SET        Total_sal=100000
WHERE      Dname='Research';
```

이 경우, `No_of_emps` 나 `Total_sal` 의 경우에는 **실제로 존재하는 Attribute가 아니기 때문에 UPDATE할 수 없다.**

- Base table에 해당 attribute가 존재하지 않는다.

이 경우에도 UPDATE하지 않는다.

UPDATING VIEW Summary

VIEW에 대한 업데이트는 성공적으로 처리될 지 보장되지 않는다.

1. VIEW가 **하나의 Base table로 정의**되어 있고, VIEW가 **Base table의 PK를 포함**하면 **그 VIEW는 Update가 가능**하다.
    - 실제로 Update가 되려면 UPDATE ... WHERE에 Condition으로 PK를 사용해야 한다.
    - PK가 존재하면 Tuple이 유일하게 결정되기 때문이다.

2. VIEW가 **여러 Base table의 JOIN으로 정의되어 있는 경우 일반적으로 Update가 불가능**하다.

3. VIEW가 **Grouping & aggregate fucntion으로 정의된 경우 업데이트가 불가능**하다.
    - Grouping에서 SELECT 문에 aggregate function을 사용하지 않고 Grouping attribute로 PK만 사용되면 가능하긴 하다.
    - Aggregate function을 사용하는 경우에는 Aggregate function이 적용된 결과가 실제로 저장되어 있지 않기 때문에 불가능하다.

장점

VIEW를 사용하지 않아도 Query로 동일한 작업을 수행할 수 있다.

- 그러나 Query가 복잡해진다.

1. VIEW만 사용하면 **사용자는 JOIN을 사용할 필요 없이 SELECT 문만 활용하여 Query를 작성할 수 있다.**
    - DB Desinger / DBA가 VIEW를 잘 구성한다면 이후 Query 작성이 쉬워진다.
    - VIEW에도 JOIN이 들어가겠지만 **사용자 입장에선 쉬워진다.**

2. **보안, 권한 설정** 측면에서도 사용될 수 있다.
    - 사람들에게 **VIEW만 보도록 하고 실제 Table은 숨길 수 있다.**
    - Physical table의 일부분만 보도록 한다.
3. Query가 주어졌을 때, 결과 Tuple을 계산하기 때문에 항상 **up-to-date**이다.

단점

1. **Performance issue**를 유발할 수 있다.
    - VIEW로 정의한 **INSERT INTO ... SELECT**문을 통해 **실제로 Physical table을 생성한 경우에 비해 성능이 저하**된다.
        - 해당 (Logical) table을 사용할 때마다 View를 통해 SELECT를 실행해 Base table에 접근하기 때문이다.
    - 특히, VIEW에 사용된 SELECT문이 복잡하고 **Time-consuming qeury일 때 성능이 악화**된다.

2. **VIEW를 통한 실제 table의 업데이트가 불가능**하다.

Index

> **빠른 Retrieval을 위한 Additional meta-data**

- EX) B-trees

`Indexing attributes` : Table을 Index하는데 사용되는 Attributes

- 해당 Attribute 값을 이용하여 정렬한 후 B-tree 등으로 이용해 Index로 사용한다.

SELECT ... WHERE에서 **WHERE에 Indexing attribute**을 사용하면 **Query 처리 속도가 빨라진다.**

Index를 다루는 명령어는 DDL이다.

CREATE INDEX

CREATE INDEX <index name> **ON** <table name> (<list of attributes>);

여러 attributes로 Index를 생성할 수 있다.

- 이 경우, **모든 Attribute의 값을 Concatnate 후 하나의 Value로 Index를 관리**한다.

```
CREATE INDEX NameIndex
ON EMPLOYEE (Lname, Fname, Minit);
```

INDEX 정렬

> **Index가 정렬되도록 하는 명령어**

ASC: 오름차순, Default 값

DESC: 내림차순

Example

```
CREATE INDEX NameIndex
ON EMPLOYEE (Lname ASC, Fname DESC, Minit ASC);
```

UNIQUE

CREATE UNIQUE INDEX <index name> **ON** <table name> (<list of attributes>);

이 경우, **<list of attributes>의 Attribute의 값이 완벽히 동일한 Tuple이 Table에 추가될 수 없도록 한다**.

- 값이 같다면 Index도 동일한데, **UNIQUE가 같은 Index 사용을 막기 때문**이다.
- Indexing attribute에 **Key constarints를 추가**하는 것과 동일하다.

CLUSTER

CREATE INDEX <index name> **ON** <table name> (<list of attributes>) **CLUSTER**;

가급적으로 **새로운 tuple은 attribute 값에 따라, 값이 가까운 기존 tuple들과 인접한 물리적 위치에 저장**되도록 한다.

- 비슷한 Index를 갖는 Tuple끼리 **실제로 물리적으로 비슷한 위치에 저장**되도록 한다.

Join이나 **Ranged condition**을 포함한 Query를 처리할 때 성능이 좋아지는 모습을 보인다.

- **JOIN**

  두 attribute에 모두 Clustered index가 있다면 **MERGE JOIN**처럼 동작할 수 있어 빠르다.

  - 가장 앞에서부터 올라가며 같은 것이 있는지 확인하는 방법이다.
  - 두 Attribute의 크기 관계에 따라 포인터를 조절한다고 생각

- **Ranged condition**

  SELECT * FROM EMPLOYEE WHERE Salary BETWEEN 40000 AND 60000; 에서 40000 ~ 60000 사이 구간을 읽는 것은 **디스크를 연속적으로 읽는 것과 동일**하다.

DROP INDEX

> **필요하지 않은 INDEX를 제거하는 명령어**

DROP INDEX <Index name>